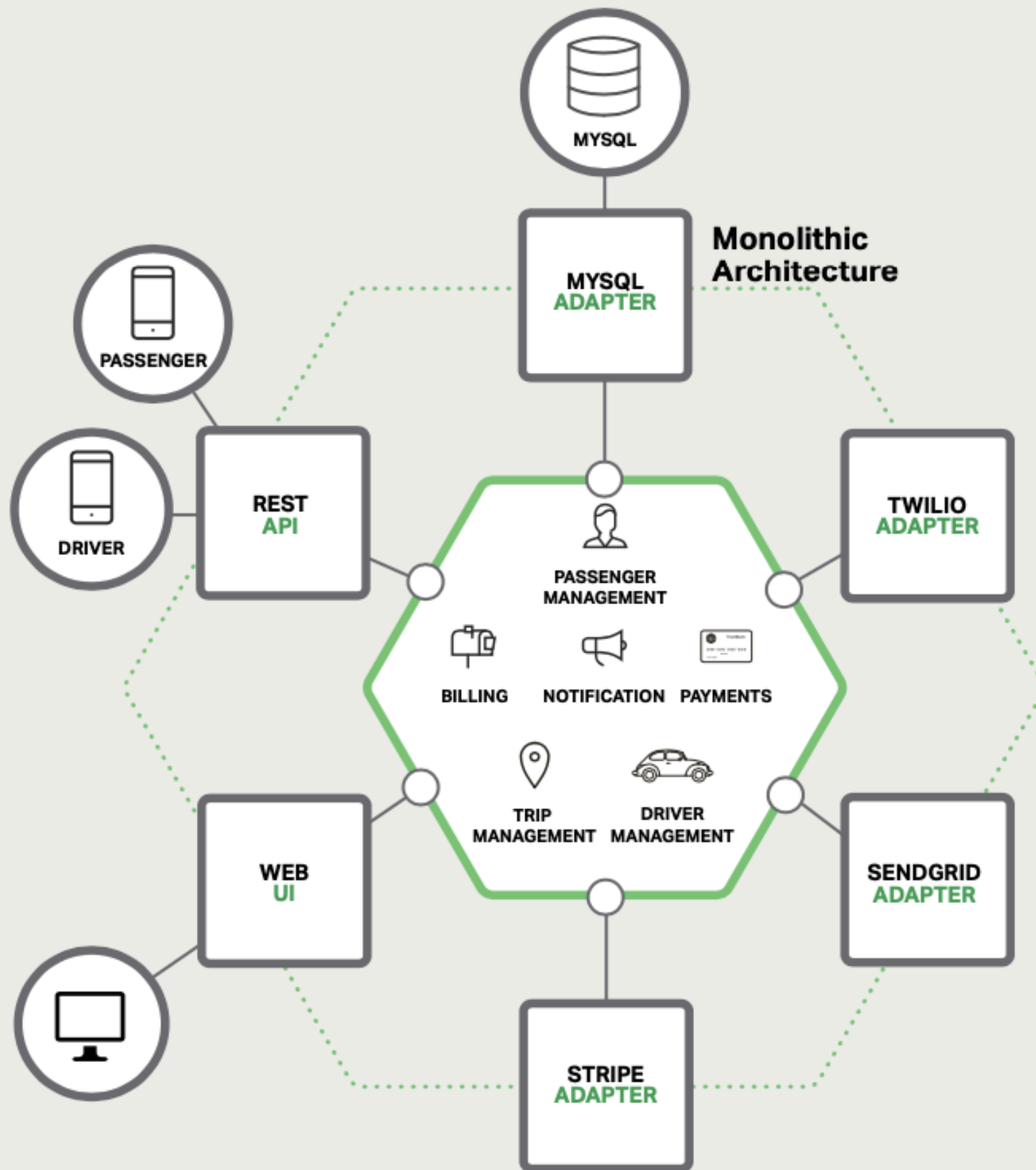


MicroServices-1

Middleware Architecture (SCS 2314)

Introduction

Monolithic Applications



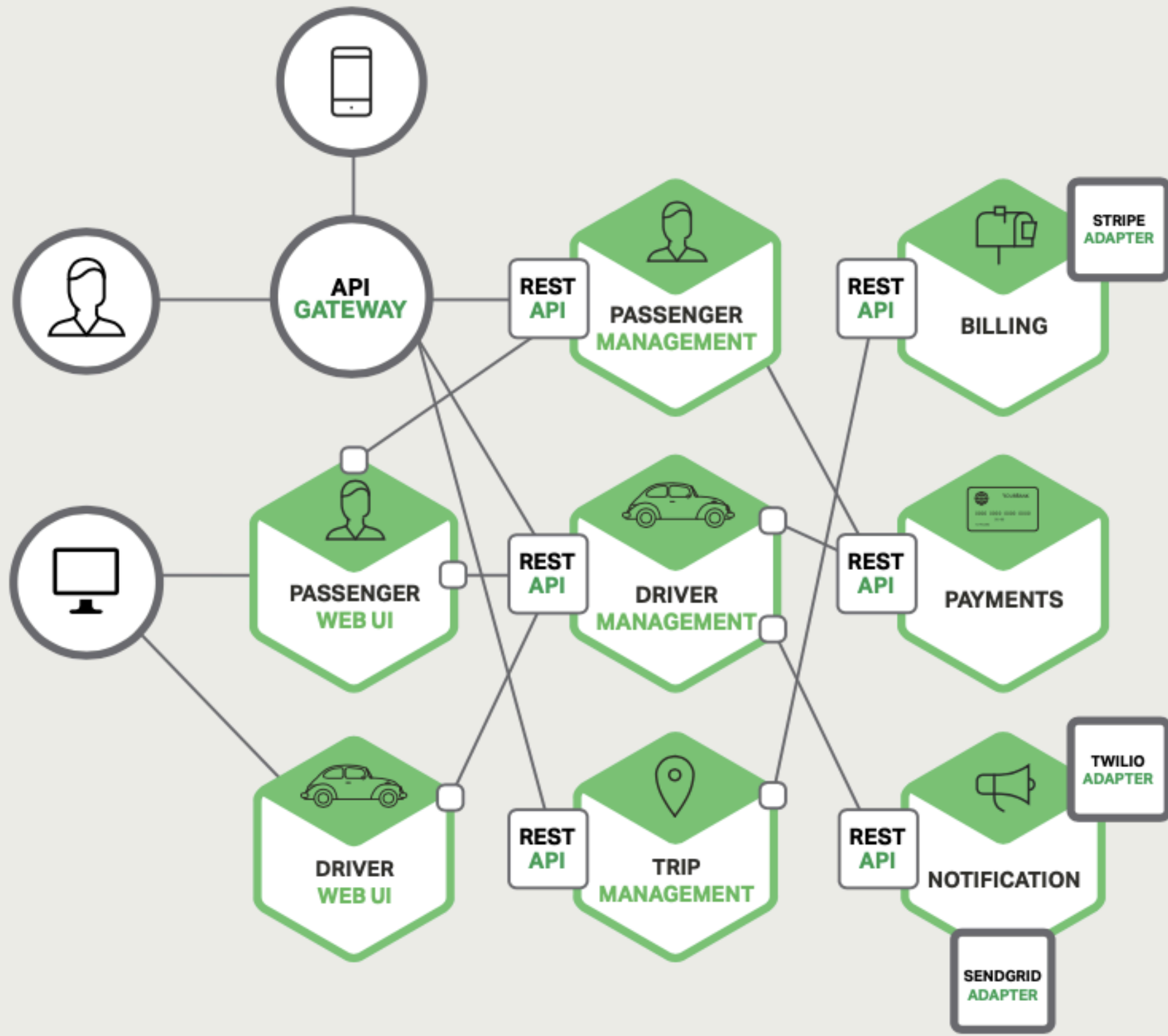
Monolithic Applications

- Despite having a **logically modular architecture**, the application is **packaged** and deployed as a **monolith**.
- The **actual format** depends on the application's language and framework.
- For example, many Java applications are packaged as **WAR files** and deployed on application servers such as Tomcat or Jetty.
- They are simple to develop since our IDEs and other tools are focused on building a single application.
- These kinds of applications are also **simple to test**. You can implement **end-to-end testing** by simply launching the application and testing the UI with a testing package such as Selenium.
- Monolithic applications are also **simple to deploy**. You just have to copy the packaged application to a server.
- You can also **scale** the application by **running multiple copies behind a load balancer**.

Drawbacks of Monolithic Applications

- One major **problem** is that the application is **overwhelmingly complex**. It's simply too large for any single developer to fully understand.
- The sheer size of the application will also **slow down development**. The larger the application, the **longer the start-up time** will be.
- Another problem with a large, complex monolithic application is that it is an **obstacle to continuous deployment**.
- Monolithic applications can also be **difficult to scale** when **different modules have conflicting resource requirements**. For example, one module might implement **CPU-intensive** image processing logic and would ideally be deployed in Amazon EC2 Compute Optimized instances. Another module might be an **in-memory database** and best suited for EC2 Memory-optimized instances.
- Another problem with monolithic applications is **reliability**. Because all modules are running within the same process, a bug in any module, such as a memory leak, can potentially **bring down the entire process**.
- Monolithic applications make it extremely **difficult to adopt new frameworks** and languages.

Microservices Applications

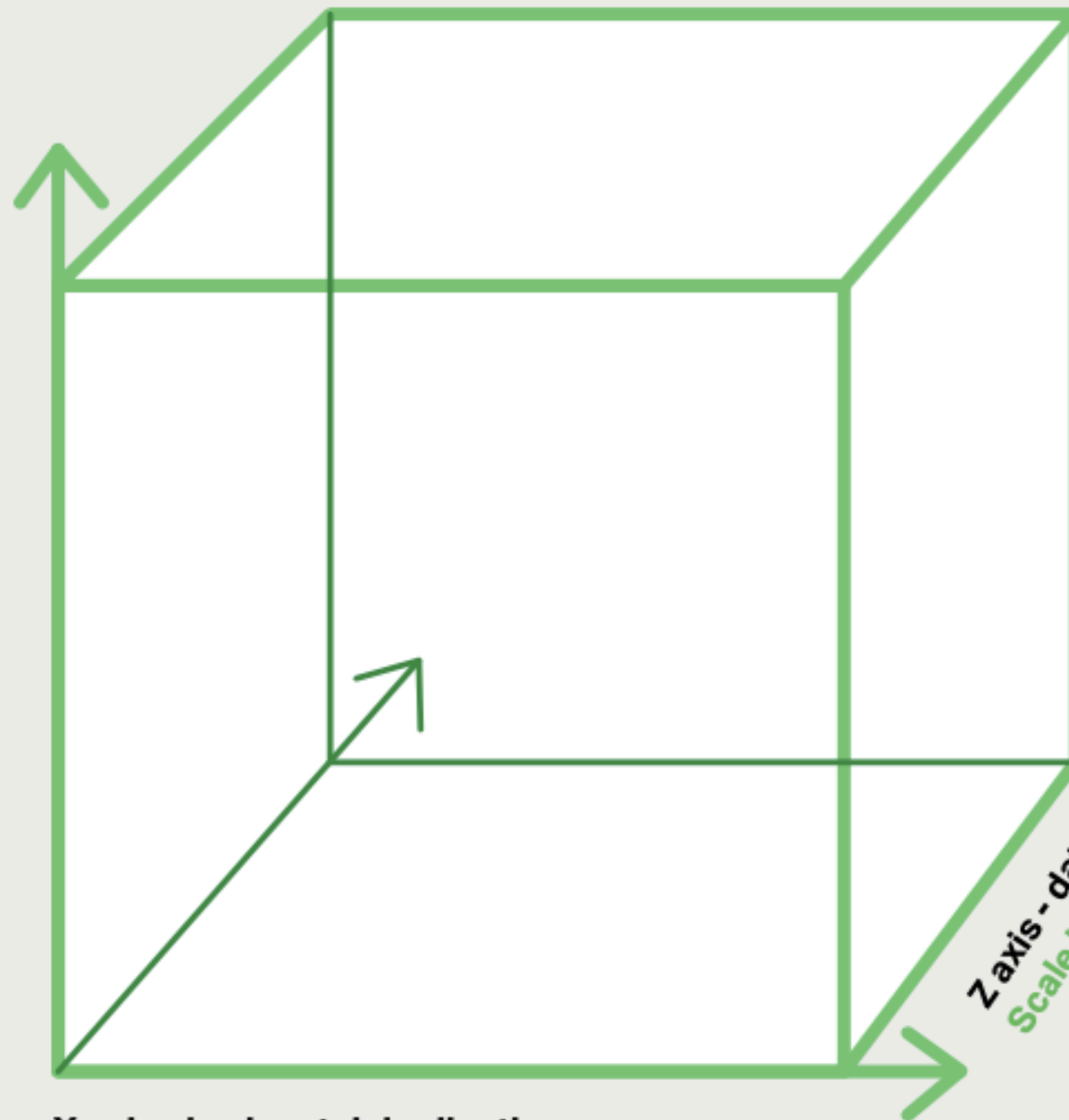


Reduce Complexity with Microservices

- A service typically implements a **set of distinct features or functionalities**, such as order management, customer management.
- Some microservices would **expose** an API that's **consumed** by other microservices or by the application's clients. Other microservices might implement a web UI at runtime, each instance is often a cloud virtual machine (VM) or a Docker container.
- Each **functional area** of the application is now implemented by its **microservice**.
- Each **backend service exposes a REST API** and most services **consume APIs provided by other services**. For example, Driver Management uses the Notification server to tell an available driver about a potential trip.
- The **UI services** invoke the other services to render web pages. Services might also use **asynchronous, message-based communication**.

Scale Cube

**Y axis -
functional
decomposition**
Scale by splitting
different things



X axis - horizontal duplication
Scale by cloning

Z axis - data partitioning
Scale by splitting similar things

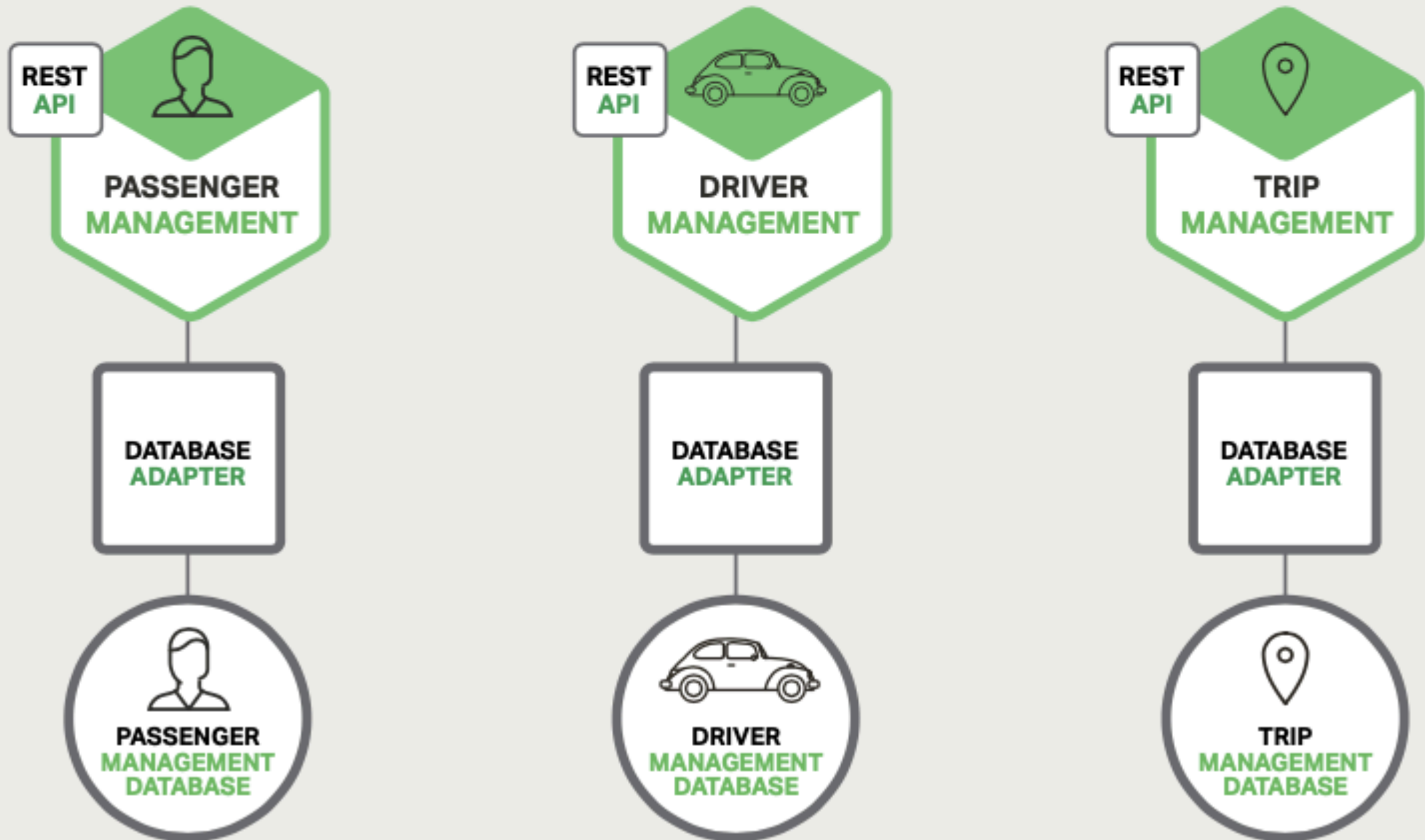
Scale Cube

- The Microservices Architecture pattern corresponds to the Y-axis scaling of the Scale Cube.
- The other two scaling axes are X-axis scaling, which consists of running multiple identical copies of the application behind a load balancer.
- Z-axis scaling (or data partitioning), where an attribute of the request (for example, the primary key of a row or the identity of a customer) is used to route the request to a particular service/server.

Application vs Database Relationship

- The Microservices Architecture pattern significantly impacts the relationship between the application and the database.
- Rather than sharing a single database schema with other services, **each service** has its **own database schema**.
- On the one hand, this approach is **at odds** with the idea of an **enterprise-wide data model**.
- It often results in **duplication** of some **data**.
- However, having a **database schema per service** is essential if you want to benefit from microservices because it ensures **loose coupling**.

Application vs Database Relationship



Microservices and SOA

- On the surface, the Microservices Architecture pattern is **similar to SOA**.
- With both approaches, the architecture **consists of a set of services**.
- However, one way to think about the Microservices Architecture pattern is that it's **SOA without the commercialisation and perceived baggage of web service specifications (WS-*) and an Enterprise Service Bus (ESB)**.
- Microservice-based applications favour **simpler, lightweight** protocols such as **REST**, rather than WS-*.
- They also very much **avoid using ESBs** and instead implement **ESB-like functionality** in the microservices themselves.
- The Microservices Architecture pattern also **rejects** other parts of SOA, such as the concept of a **canonical schema** for data access.

Benefits of Microservices

- Microservices **tackles the problem of complexity**.
 - It **decomposes** what would otherwise be a monstrous monolithic application **into a set of services**.
 - While the total amount of functionality is unchanged, the application has been broken up into **manageable chunks or services**.
 - Each service has a **well-defined boundary** in the form of a remote procedure call (RPC)-driven or message-driven API.
- This architecture enables **each service to be developed independently** by a **team** that is focused on that service.
 - The developers are **free to choose** whatever **technologies** make sense, provided that the service honours the API contract

Benefits of Microservices

- The Microservices Architecture pattern enables each microservice to be **deployed independently**.
 - Developers **never need to coordinate the deployment** of changes that are local to their service.
 - The Microservices Architecture pattern makes **continuous deployment possible**.
- The Microservices Architecture pattern enables **each service to be scaled independently**.
 - You can deploy just the number of instances of each service that satisfy its **capacity** and **availability** constraints.
 - For example, you can deploy a **CPU-intensive** image processing service on EC2 Compute Optimized instances and deploy an in-memory database service on EC2 **Memory-optimized** instances.

Drawbacks of Microservices

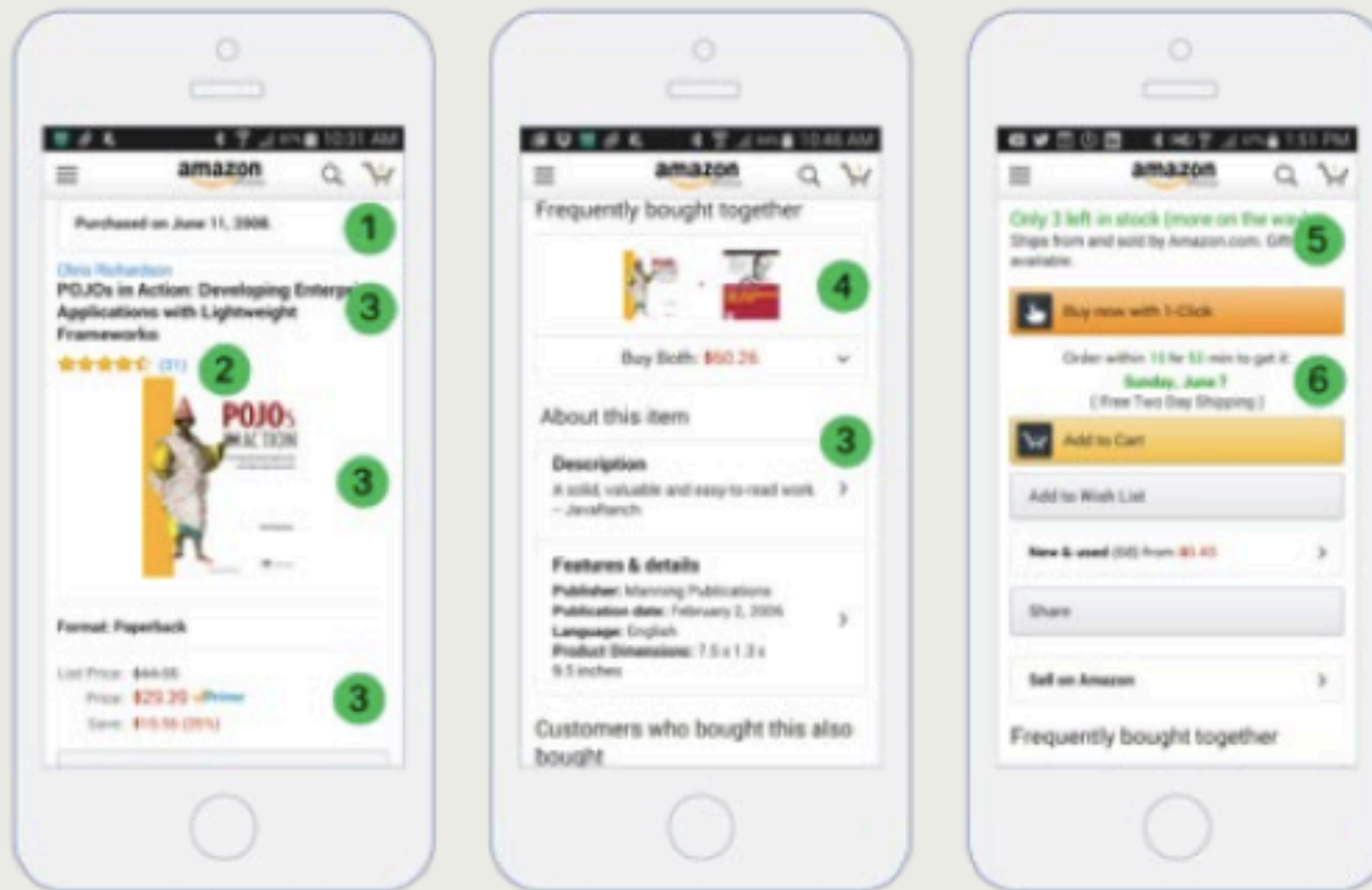
- While small services are preferable, it's important to remember that small services are a means to an end and not the primary goal. The goal of microservices is to sufficiently decompose the application to facilitate agile application development and deployment.
- Another major drawback of microservices is the complexity that arises from the fact that a microservices application is a distributed system.
 - Developers need to choose and implement an inter-process communication mechanism based on either messaging or RPC.
 - Moreover, they must also write code to handle partial failure, since the destination of a request might be slow or unavailable. It's much more complex than in a monolithic application, where modules invoke one another via language-level method/procedure calls.
- Another challenge with microservices is the partitioned database architecture. Business transactions that update multiple business entities are fairly common.
- Testing a microservices application is also much more complex.

Drawbacks of Microservices

- Another major challenge with the Microservices Architecture pattern is **implementing changes that span multiple services**.
- **Deploying a microservices-based application is also much more complex.**
 - Each service will have **multiple runtime instances**. That's many more moving parts that need to be configured, deployed, scaled, and monitored.
 - One approach to automation is to use an **off-the-shelf platform-as-a-service (PaaS)** such as Cloud Foundry. A PaaS provides developers with an easy way to deploy and manage their microservices. It insulates them from concerns such as procuring and configuring IT resources.
 - Another way to automate the deployment of microservices is to **develop your own PaaS**. One typical starting point is to use a clustering solution, such as Kubernetes, in conjunction with container technology such as Docker.

Using an API Gateway

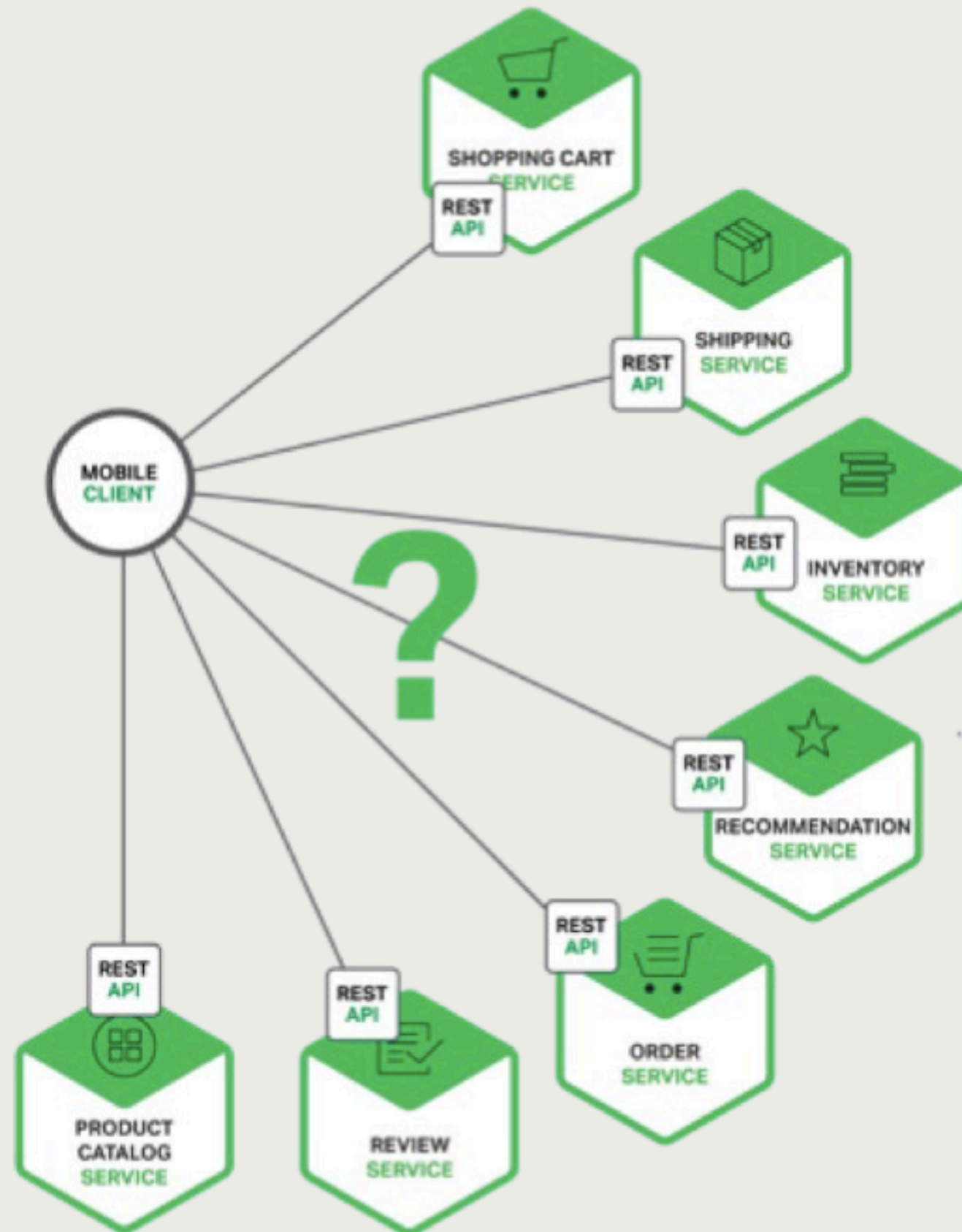
Shopping Application



Monolithic Approach

- When using a monolithic application architecture, a mobile client retrieves this data by making a **single REST call** to the application, such as:
- **GET: api.company.com/productdetails/productId**
- A **load balancer routes** the request to one of several identical application instances.
- The application then **queries** various database tables and **returns the response** to the client

Direct Client to Microservice Approach



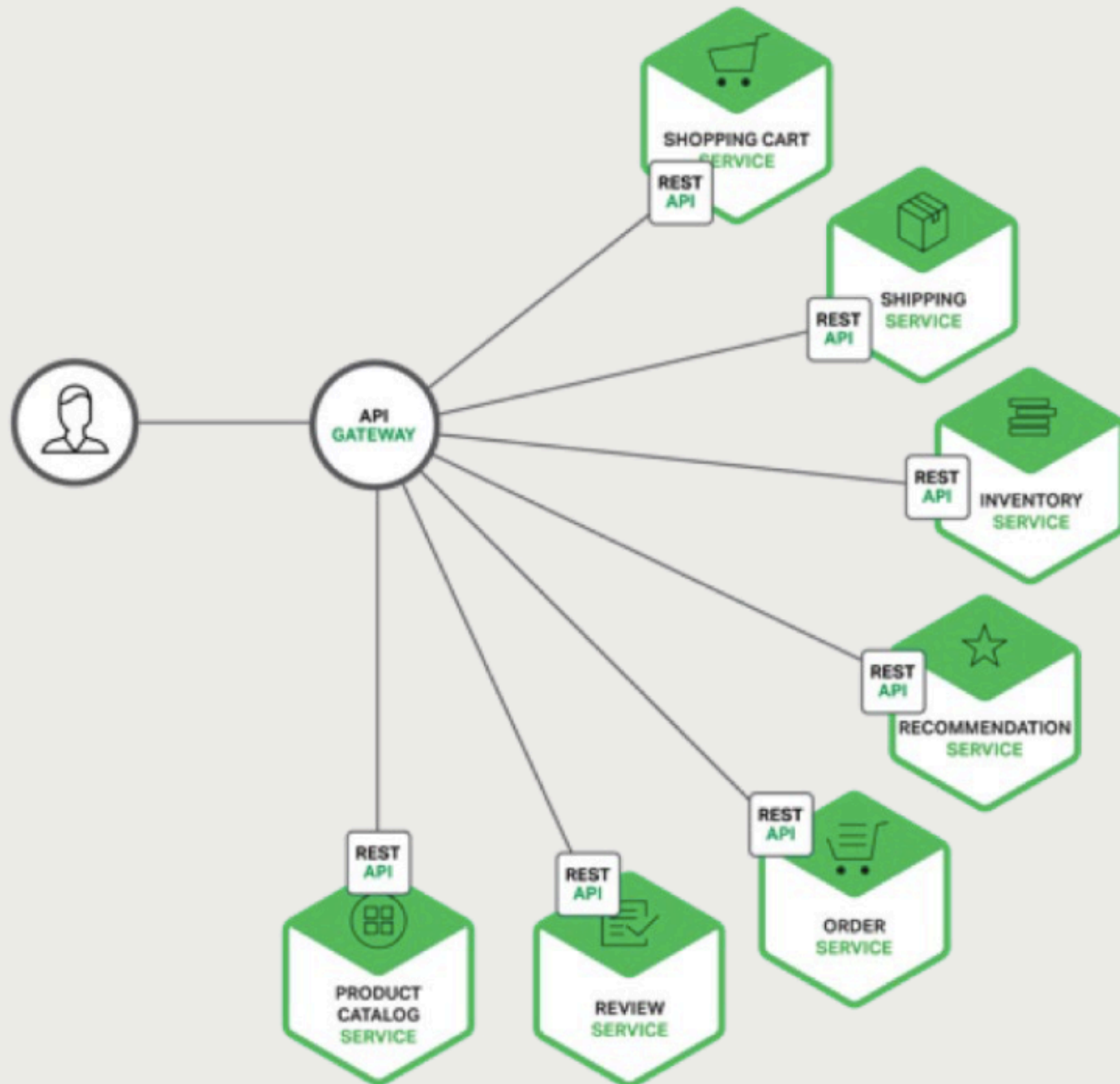
Direct Client to Microservice

- In theory, a client could make requests to each of the microservices **directly**. Each microservice would have a public endpoint:
 - **`https://serviceName.api.company.name`**
- This URL would map to the **microservice's load balancer**, which distributes requests across the available instances.
- To retrieve the product-specific page information, the mobile client would make requests to each of the services listed above.

Drawbacks - Direct Client to Microservice

- One problem is the mismatch between the needs of the client and the **fine-grained APIs** exposed by each of the microservices. **The client may have to make several separate requests.**
- Another problem is the **use of protocols that are not web-friendly.**
 - One service might use **Thrift binary RPC** while another service might use the **AMQP** messaging protocol. Neither protocol is particularly browser or firewall-friendly and is best used internally.
 - An application should use protocols such as **HTTP** and WebSocket outside of the firewall.
- **Difficult to refactor the microservices.** Over time we might want to change how the system is partitioned into services.

Using API Gateway for Client-Service



Using API Gateway for Client-Service

- An **API Gateway** is a server that is the **single entry point** into the system. It is similar to the **Facade pattern** from object-oriented design.
- The API Gateway **encapsulates the internal system architecture** and provides an API that is tailored to each client.
- It might have **other responsibilities** such as **authentication, monitoring, load balancing, caching**, request shaping and management, and static response handling.

API Gateway

- The API Gateway is responsible for request **routing, composition, and protocol translation**.
- All requests from clients first **go through the API Gateway**. It then **routes** requests to the appropriate **microservice**.
- The API Gateway will often handle a request by invoking **multiple microservices** and **aggregating the results**.
- It can **translate** between web protocols such as HTTP and WebSocket and web-unfriendly protocols that are used internally.

API Gateway - Benefits and Drawbacks

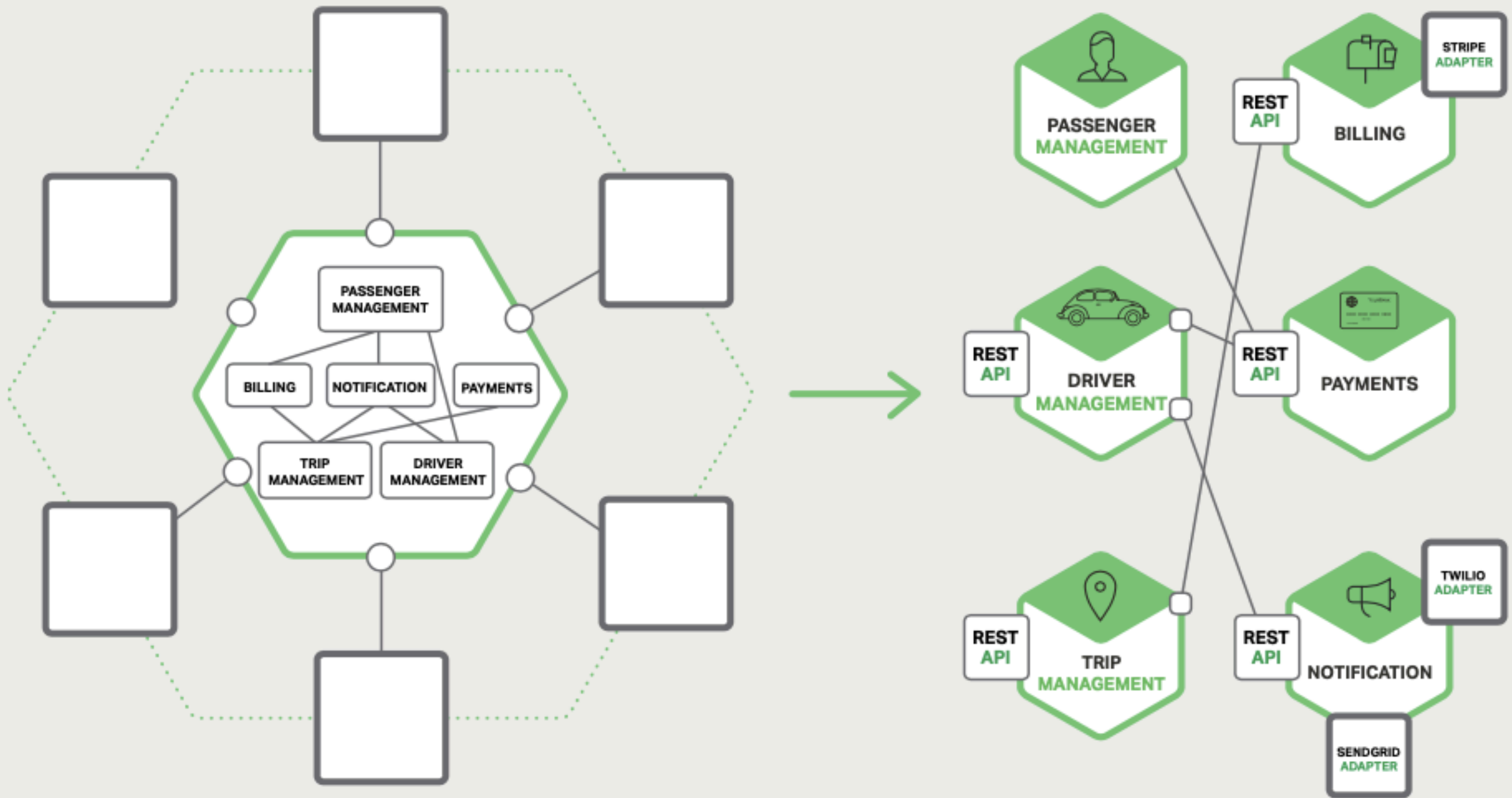
- A major benefit of using an API Gateway is that it **encapsulates the internal structure of the application**.
 - Rather than having to invoke specific services, clients simply **talk to the gateway** - The API gateway hides the **complexity** of microservices from the client.
 - The API Gateway provides each kind of client with a **specific API**. This reduces the number of round trips between the client and application. It also **simplifies the client code**.
- It is yet another **highly available component** that must be developed, deployed, and managed.
 - There is also a risk that the API Gateway becomes a **development bottleneck**.
 - Developers must **update** the API Gateway to expose each microservice's endpoints.

Inter-Process Communication

IPC - Monolithic vs Microservices

- In a monolithic application, components invoke one another via language-level **method or function calls**.
- In contrast, a micro-services-based application is a distributed system running on **multiple machines/services**.
- Each **micro-service instance** is typically a **process**.

IPC - Monolithic vs Microservices



Client-Service Interaction Styles

- There are several client-service interaction styles.
- Dimension 1: Number of participants
 - **One-to-one** – Each client request is processed by exactly one service instance.
 - **One-to-many** – Each request is processed by multiple service instances.
- Dimension 2: Communication Mode
 - **Synchronous**–The client expects a timely response from the service and might even block while it waits
 - **Asynchronous** – The client doesn't block while waiting for a response, and the response, if any, isn't necessarily sent immediately.

Client-Service Interaction Styles

	ONE-TO-ONE	ONE-TO-MANY
SYNCHRONOUS	Request/response	—
ASYNCHRONOUS	Notification	Publish/subscribe
	Request/async response	Publish/async responses

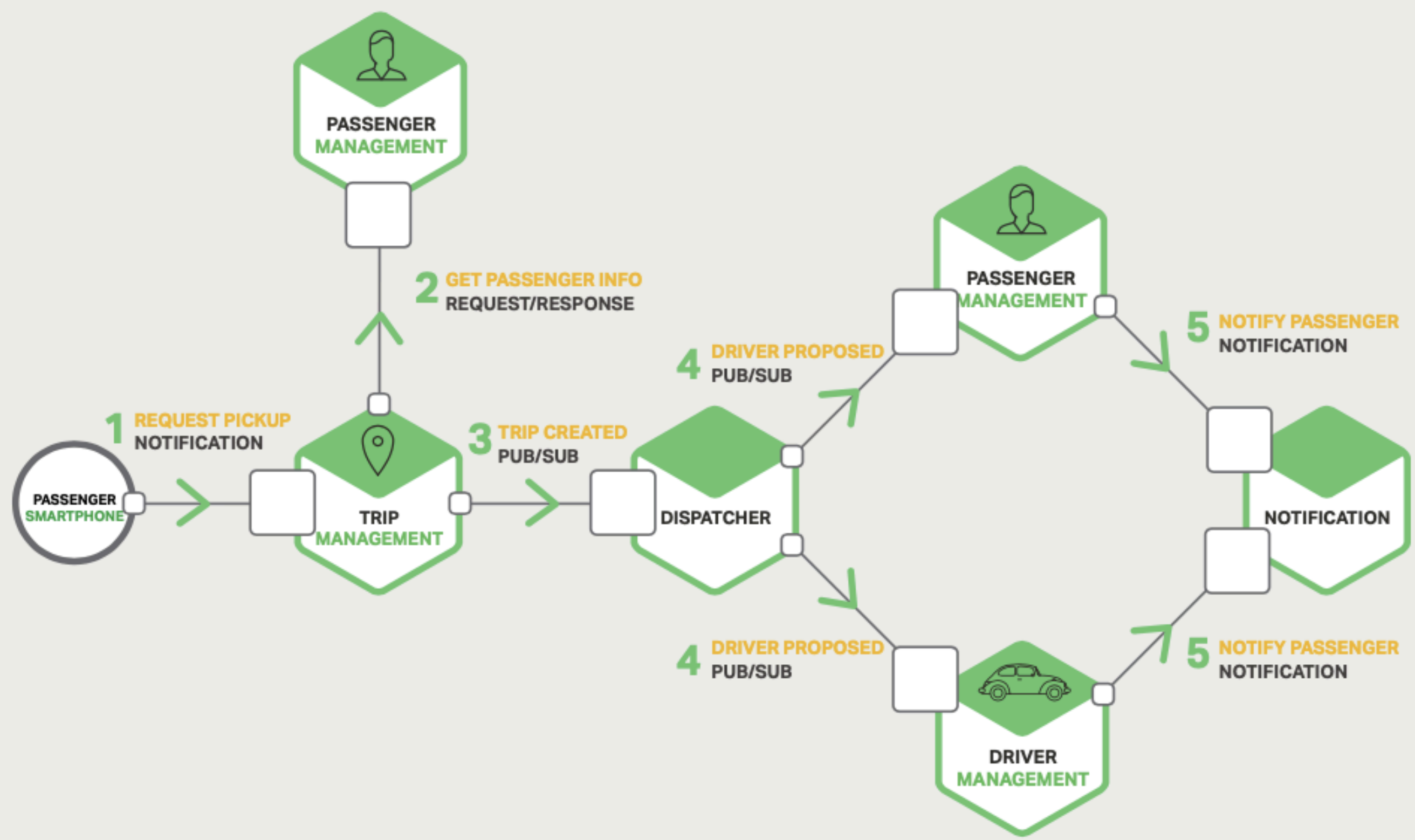
Client-Service Interaction Styles

- There are the following kinds of **one-to-one** interactions:
 - **Request/response(sync)** – A client makes a request for a service and waits for a response.
 - The client expects the response to arrive in a timely fashion.
 - In a thread-based application, the thread that makes the request might even be blocked while waiting.
 - **Notification(async)** (a.k.a. a one-way request) – A client sends a request to a service but no reply is expected or sent.
 - **Request/async response** – A client sends a request to a service, which replies asynchronously.
 - The client does not block while waiting and is designed with the assumption that the response might not arrive for a while.

Client-Service Interaction Styles

- There are the following kinds of **one-to-many** interactions, both of which are **asynchronous**:
 - **Publish/subscribe** – A client publishes a notification message, which is consumed by zero or more interested services.
 - **Publish/async responses** – A client publishes a request message and then waits a certain amount of time for responses from interested services.

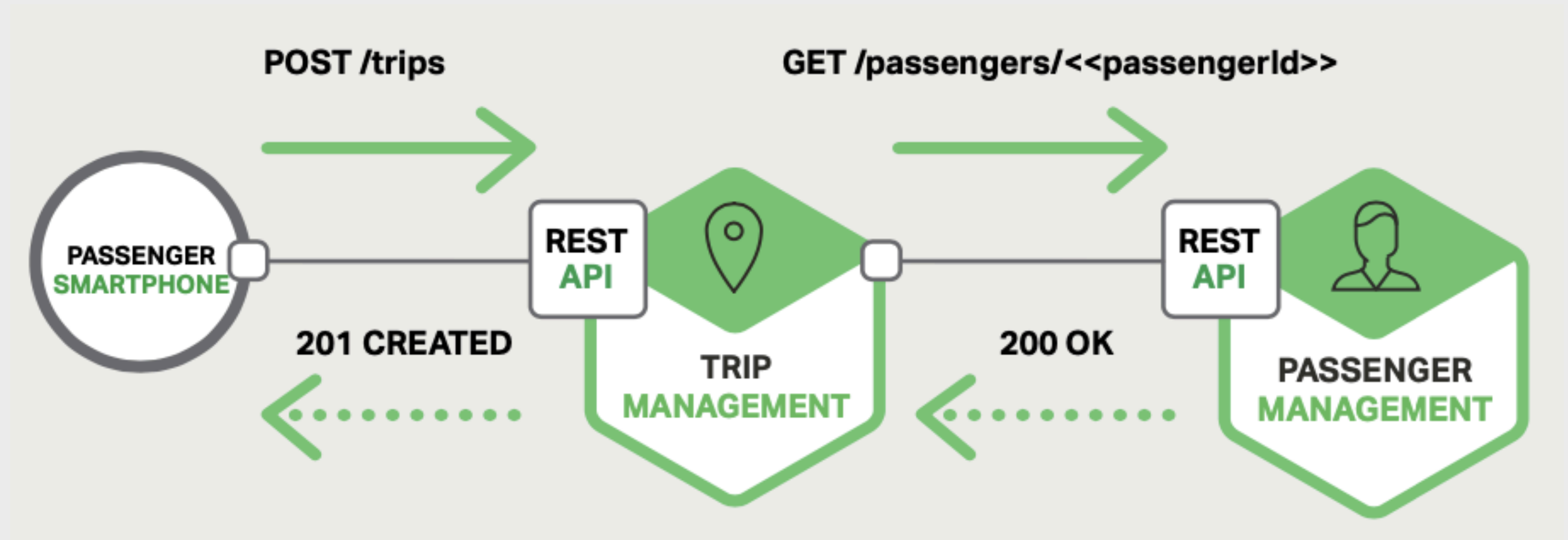
Using Multiple IPC Mechanisms for Service Interactions



Synchronous, Request/Response IPC

- When using a **synchronous**, request/response-based IPC mechanism, a client sends a request to a service. The service processes the request and sends back a response.
- In many clients, the thread that makes the request **blocks** while waiting for a response. However, unlike when using messaging, the client assumes that the response will arrive in a timely fashion.
- There are numerous protocols to choose from. Two popular protocols are **REST** and **Thrift**.

Synchronous, Request/Response IPC



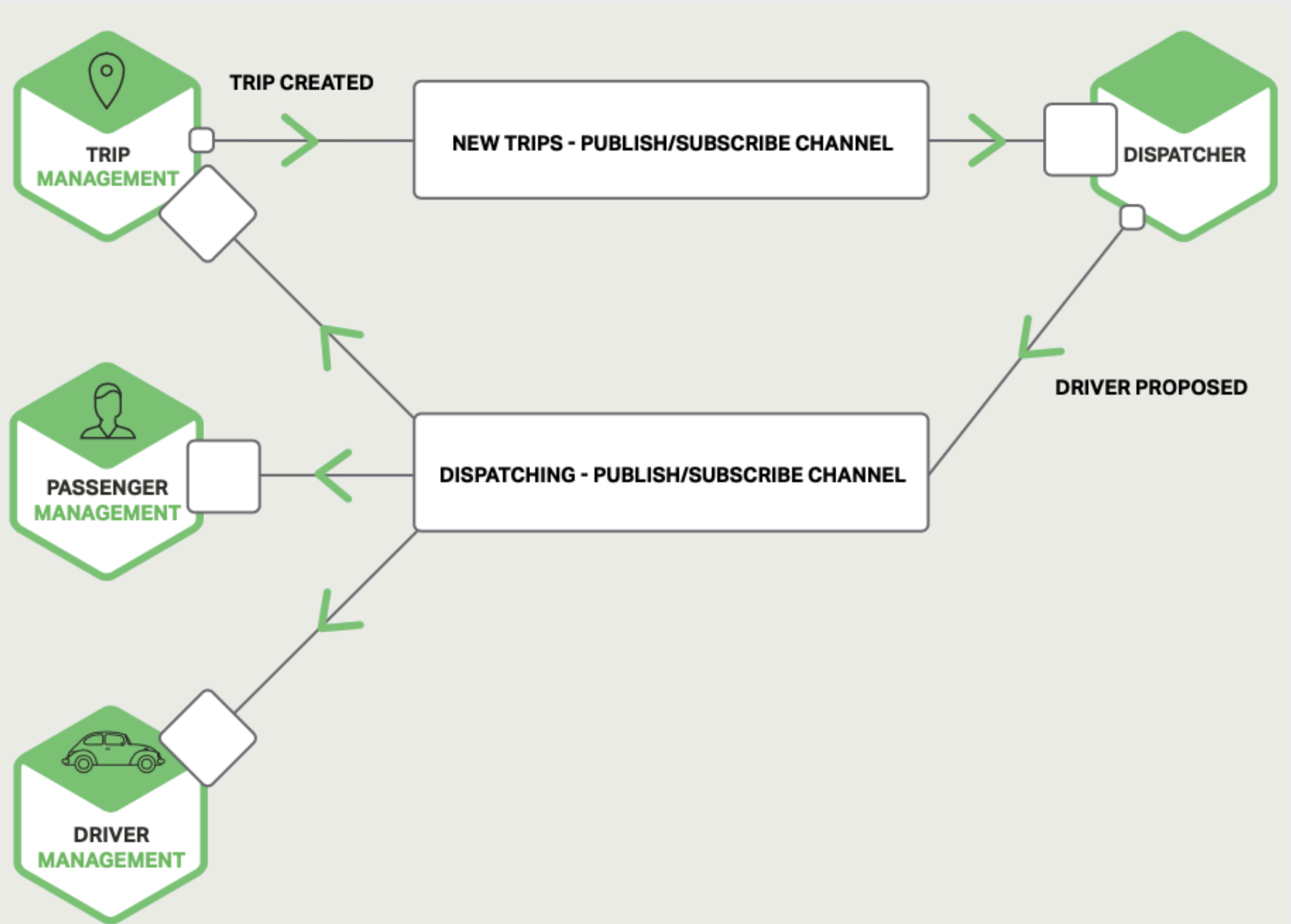
Asynchronous Message-Based Communication

- A client makes a request to a service by sending it a message.
- If the service is expected to reply, it does so by sending a separate message back to the client.
- Since the communication is asynchronous, the client **does not block waiting** for a reply. Instead, the client is developed/coded assuming that the reply will not be received immediately.

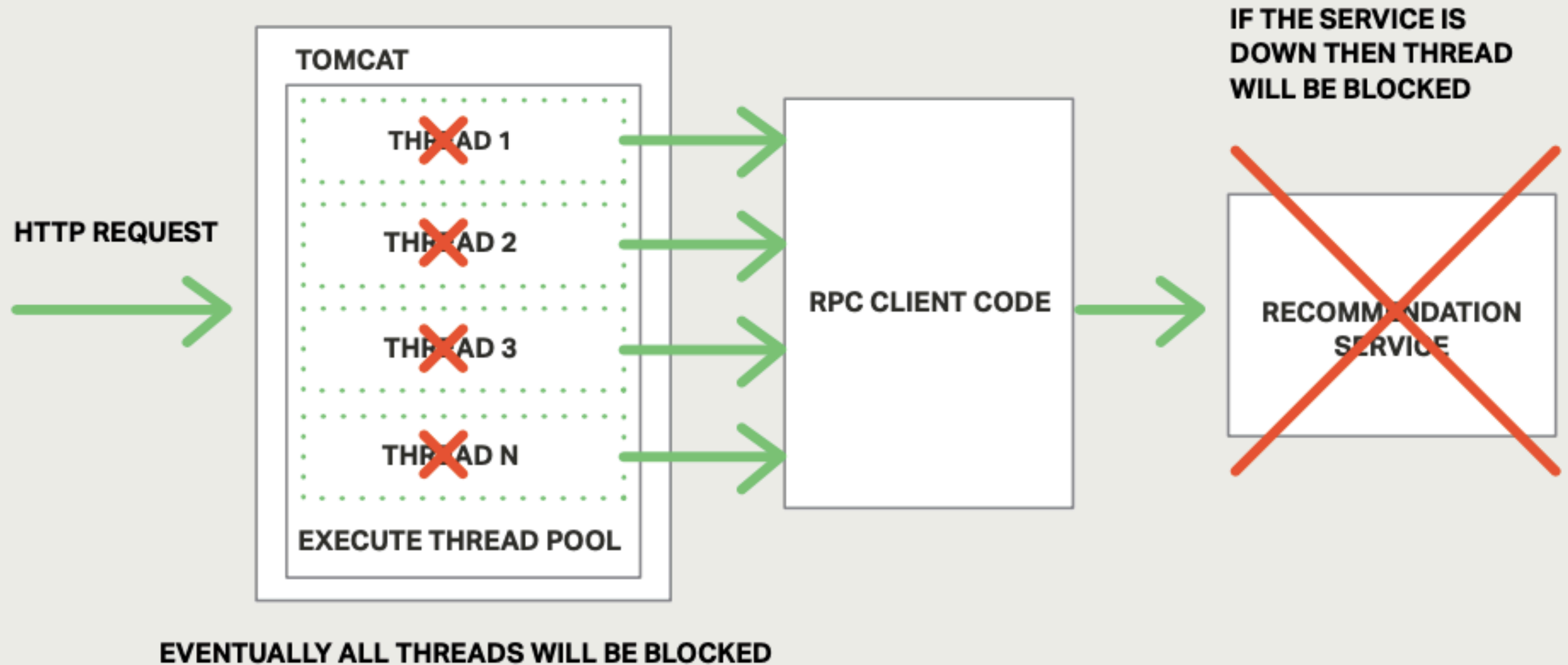
Asynchronous Message-Based Communication

- Messages are exchanged over **channels**. Any number of **producers** can send messages to a channel. Similarly, any number of **consumers** can receive messages from a channel.
- There are two kinds of channels:
 - **Point-to point channel** delivers a message to exactly one of the consumers that are reading from the channel. Services use point-to-point channels for the **one-to-one** interaction styles described earlier.
 - **Publish-subscribe channel** delivers each message to all of the attached consumers. Services use publish-subscribe channels for the **one-to-many** interaction styles described above.
- There are a large number of open source messaging systems/ middleware to choose from, including **RabbitMQ**, Apache **Kafka**, Apache **ActiveMQ** and **NSQ**.

Asynchronous Message-Based Communication



Handling Partial Failures



Strategies for Handling Partial Failures

- **Network timeouts** – Never block indefinitely and always use timeouts when waiting for a response. Using timeouts ensures that resources are never tied up indefinitely.
- **Limiting the number of outstanding requests** – Impose an upper bound on the number of outstanding requests that a client can have with a particular service. If the limit has been reached, it is probably pointless to make additional requests, and those attempts need to fail immediately.
- **Circuit breaker pattern** – Track the number of successful and failed requests. If the error rate exceeds a configured threshold, trip the circuit breaker so that further attempts fail immediately. If a large number of requests are failing, that suggests the service is unavailable and that sending requests is pointless. After a timeout period, the client should try again and, if successful, close the circuit breaker.
- **Provide fallbacks** – Perform fallback logic when a request fails. For example, return cached data or a default value, such as an empty set of recommendations.

IPC Technologies

- There are lots of different IPC technologies to choose from. Services can use **synchronous request/response-based** communication mechanisms such as **HTTP-based REST**.
- Alternatively, they can use **asynchronous**, message-based communication mechanisms such as **AMQP** or **STOMP**.
- There are also a variety of different message formats. Services can use human-readable, text-based formats such as **JSON** or **XML**.
- Alternatively, they can use a **binary format** (which is more efficient) such as **Avro** or **Protocol Buffers**.

Evolving APIs

- Sometimes, you must make major, **incompatible changes** to an API.
- Since you can't force clients to upgrade immediately, a **service must support older versions** of the API for some period of time.
- If you are using an HTTP-based mechanism such as REST, **one approach is to embed the version number in the URL**. Each service instance might handle multiple versions simultaneously.
- Alternatively, you could **deploy different instances** that each handle a particular version.

Evolving APIs

- A service's API invariably **changes over time**.
- How you handle an API change depends on the **nature of the change**.
- Some changes are **minor** and backwards compatible with the previous version - for example, **adding attributes to requests or responses**.
 - It makes sense to design clients and services so that they observe the robustness principle.
 - Clients that use an **older API** should continue to work with the new version of the service.
- The service provides **default values** for the missing **request attributes** and the clients ignore any extra response attributes.

Service Registration and Discovery

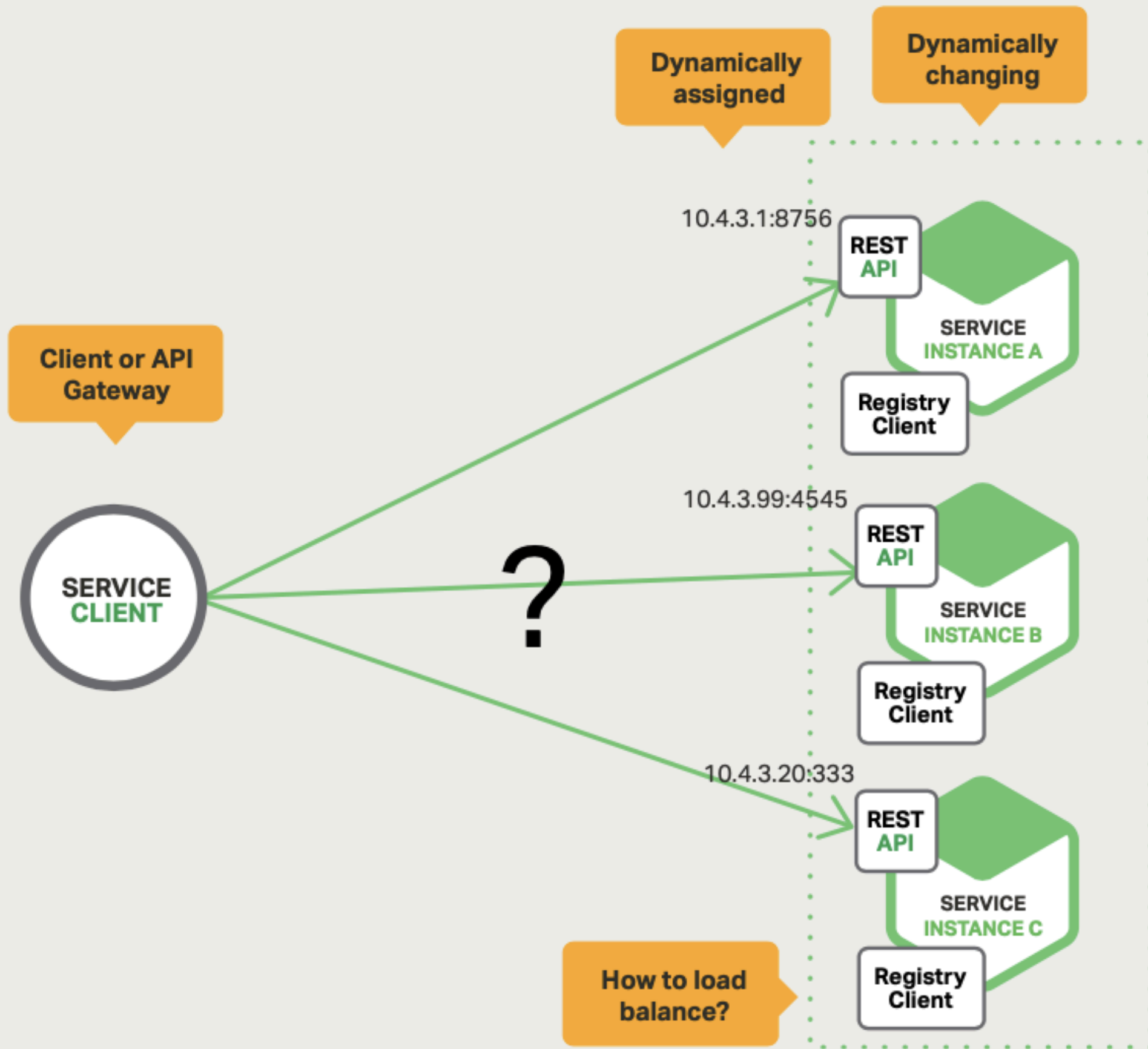
API Gateway Implementation Concerns

- Performance and Scalability
- Using a reactive programming model
- Service Invocation
- Service Discovery
- Handling partial failures

Nature of Service Discovery

- Traditionally, network services are located using the network location (**IP address and Port**) of a service instance.
- In a traditional application running on physical hardware, the network locations of service instances are relatively **static**.
- In a modern, cloud-based **microservices** application, service **instances are dynamically assigned** network locations.
- Moreover, the set of **service instances changes dynamically** because of **autoscaling, failures, and upgrades**.
- Consequently, your **client code** needs to use a more elaborate **service discovery** mechanism.

Nature of Service Discovery



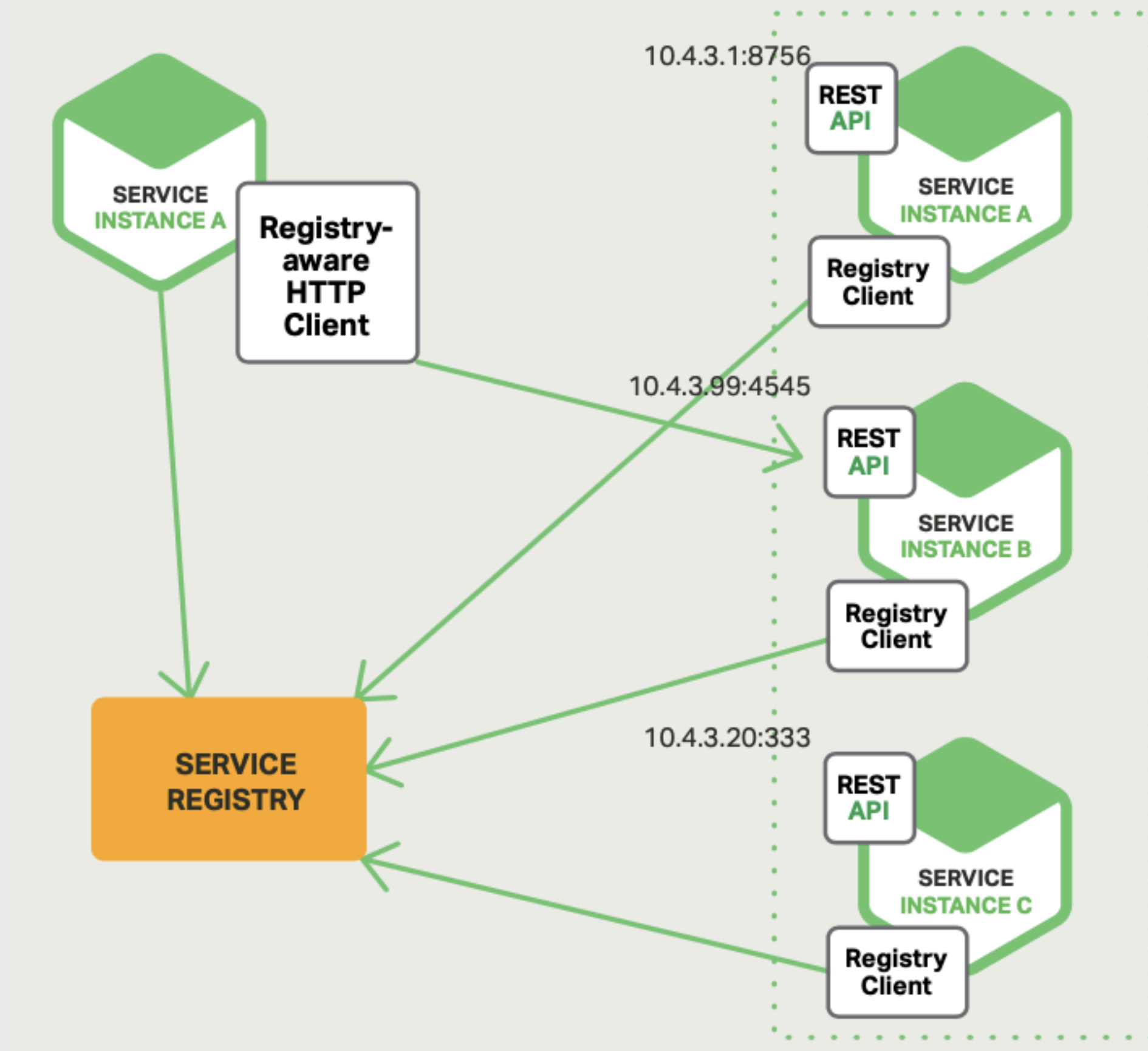
Service Discovery Patterns

- There are two main service discovery patterns:
 - **Client-side** discovery
 - **Server-side** discovery
- The network location of a **service instance** is **registered** with the **service registry** when it starts up.
- It is **removed** from the **service registry** when the instance terminates.
- The service instance's registration is typically **refreshed periodically** using a **heartbeat** mechanism.

Client-Side Service Discovery

- The **client is responsible** for determining the network locations of available service instances and load-balancing requests across them.
- The **client queries a service registry**, which is a database of available service instances.
- The **client** then uses a **load-balancing algorithm** to select one of the available service instances and makes a request.

Client-Side Service Discovery



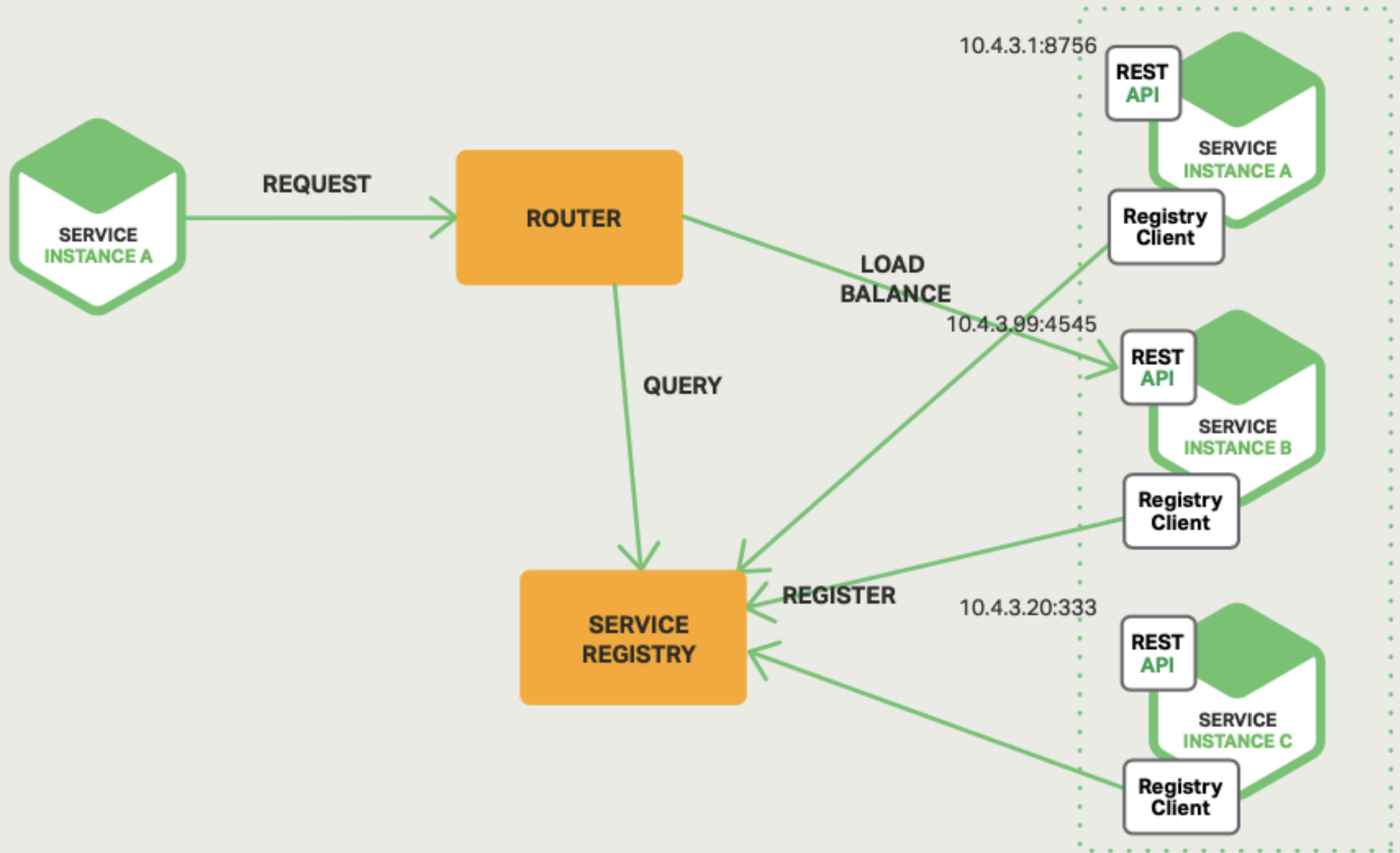
Client-Side Service Discovery

- The client-side discovery pattern has a variety of **benefits** and **drawbacks**.
- This pattern is **relatively straightforward** and, except for the service registry, there are no other moving parts.
- Since the **client knows about the available service instances**, it can make intelligent, application-specific load-balancing **decisions** such as using **hashing** consistently.
- One significant drawback of this pattern is that it **couple**s the client with the service registry. You must implement **client-side service discovery logic** for each programming language and framework used by your service clients.

Server-Side Service Discovery

- The client makes a request for a **service** via a **load balancer**.
- The **load balancer** queries the **service registry** and **routes** each request to an **available service instance**.
- As with client-side discovery, service instances are **registered** and **deregistered** with the **service registry**.
- The AWS Elastic Load Balancer (**ELB**) is an example of a **server-side discovery router**.
 - ELB is commonly used to load balance external traffic from the Internet. **ELB** can be used to load balance traffic that is internal to a virtual private cloud (**VPC**).

Server-Side Service Discovery



Server-Side Service Discovery

- A **client** makes **requests** (HTTP or TCP) via the ELB using its **DNS** name.
- The **ELB** load balances the traffic among a set of registered Elastic Compute Cloud (**EC2**) instances or EC2 Container Service (ECS) containers.
- There isn't a separately visible service registry. Instead, EC2 instances and ECS containers are registered with the ELB itself.
- **HTTP** servers and load balancers can also be used as **server-side discovery load balancers**.
- One great benefit of this pattern is that **details of discovery are abstracted away from the client**.
 - Clients simply make requests to the load balancer. This eliminates the need to implement discovery logic for each programming language and framework used by your service clients.
 - Some deployment environments provide this functionality as a standard (AWS).
- This pattern also has drawbacks. Unless the load balancer is provided by the deployment environment, it is yet another **highly available system component** that you need to set up and manage.

Service Registry

- The service registry is a key part of service discovery.
- It is a **database containing the network locations of service instances**.
- A service registry needs to be **highly available** and up to date.
- **Clients can cache** network locations obtained from the service registry. However, that information eventually becomes out of date and clients become unable to discover service instances.
- A service registry consists of a **cluster of servers** that use a replication protocol to maintain consistency.

Service Registry - Netflix Eureka

- **Netflix Eureka** is a good example of a service registry.
- It provides a **REST API** for **registering** and **querying** service instances.
- A service instance **registers** its network location using a **POST** request.
- Every 30 seconds it must **refresh** its registration using a **PUT** request.
- A registration is **removed** by either using an HTTP **DELETE** request or by the instance registration timing out.
- As you might expect, a **client can retrieve** the registered service instances by using an HTTP **GET** request.

Service Registry - Examples

- Other examples of service registries include:
 - **etcd** – A highly available, distributed, consistent, key-value store that is used for shared configuration and service discovery. Two notable projects that use etcd are Kubernetes and Cloud Foundry.
 - **Consul** – A tool for discovering and configuring services. It provides an API that allows clients to register and discover services. Consul can perform health checks to determine service availability.
 - **Apache ZooKeeper** – A widely used, high-performance coordination service for distributed applications. Apache ZooKeeper was originally a subproject of Hadoop but is now a separate, top-level project.

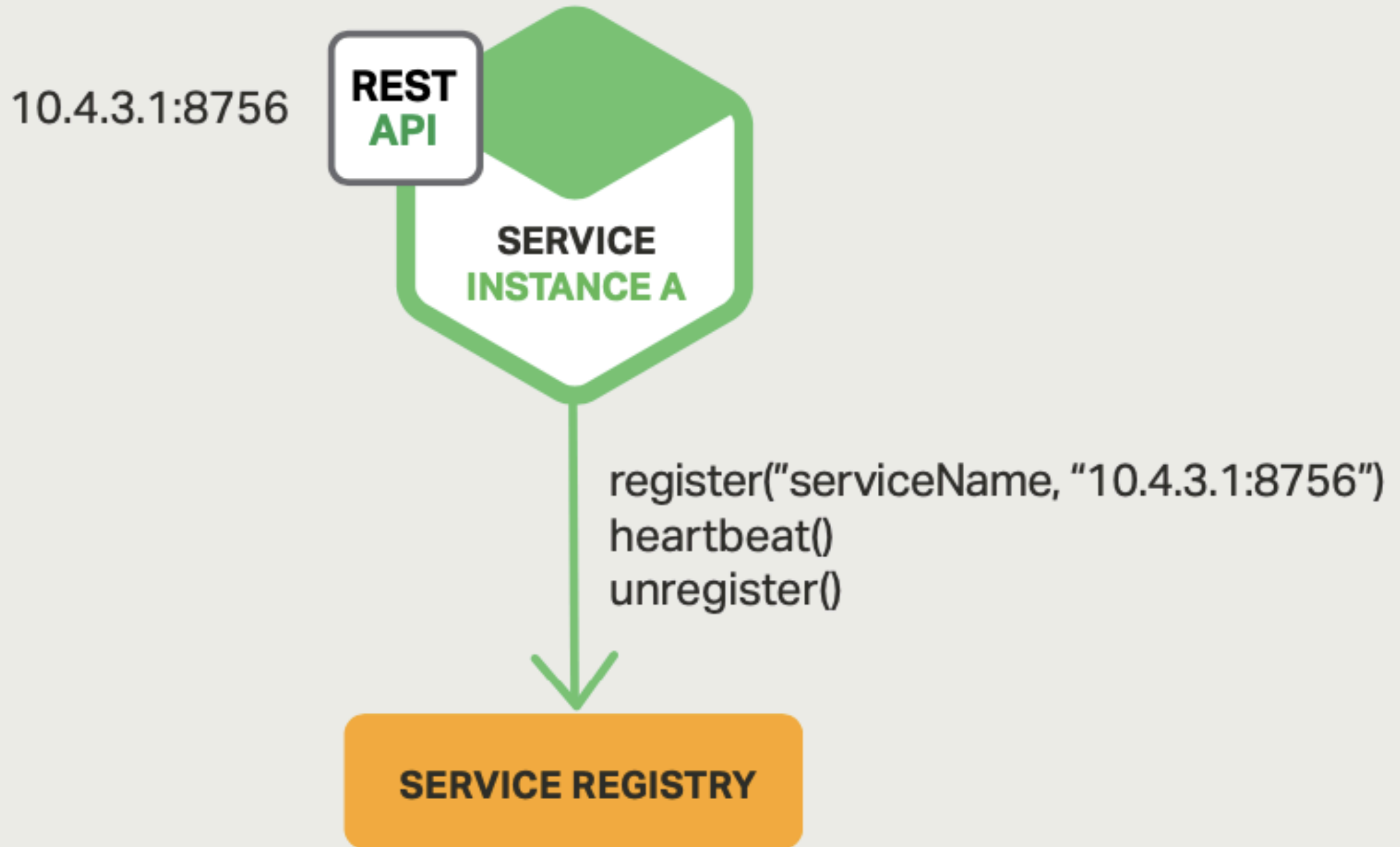
Service Registration Options

- Service instances must be registered with and unregistered from the service registry.
- There are a couple of different ways to handle the registration and unregistration.
 - **Self-registration**: One option is for service instances to register themselves, the self-registration pattern.
 - **3rd Party Registration**: The other option is for some other system component to manage the registration of service instances, the third-party registration pattern.

Self-Registration Pattern

- When using the self-registration pattern, a **service instance is responsible for registering and unregistering** itself with the service registry.
- If required, a **service instance sends heartbeat** requests to prevent its registration from **expiring**.
- One benefit is that it is relatively **simple** and doesn't require any other system components.
- A major drawback is that it couples the service instances to the service registry. You must **implement the registration code in each programming language and framework** used by your services.

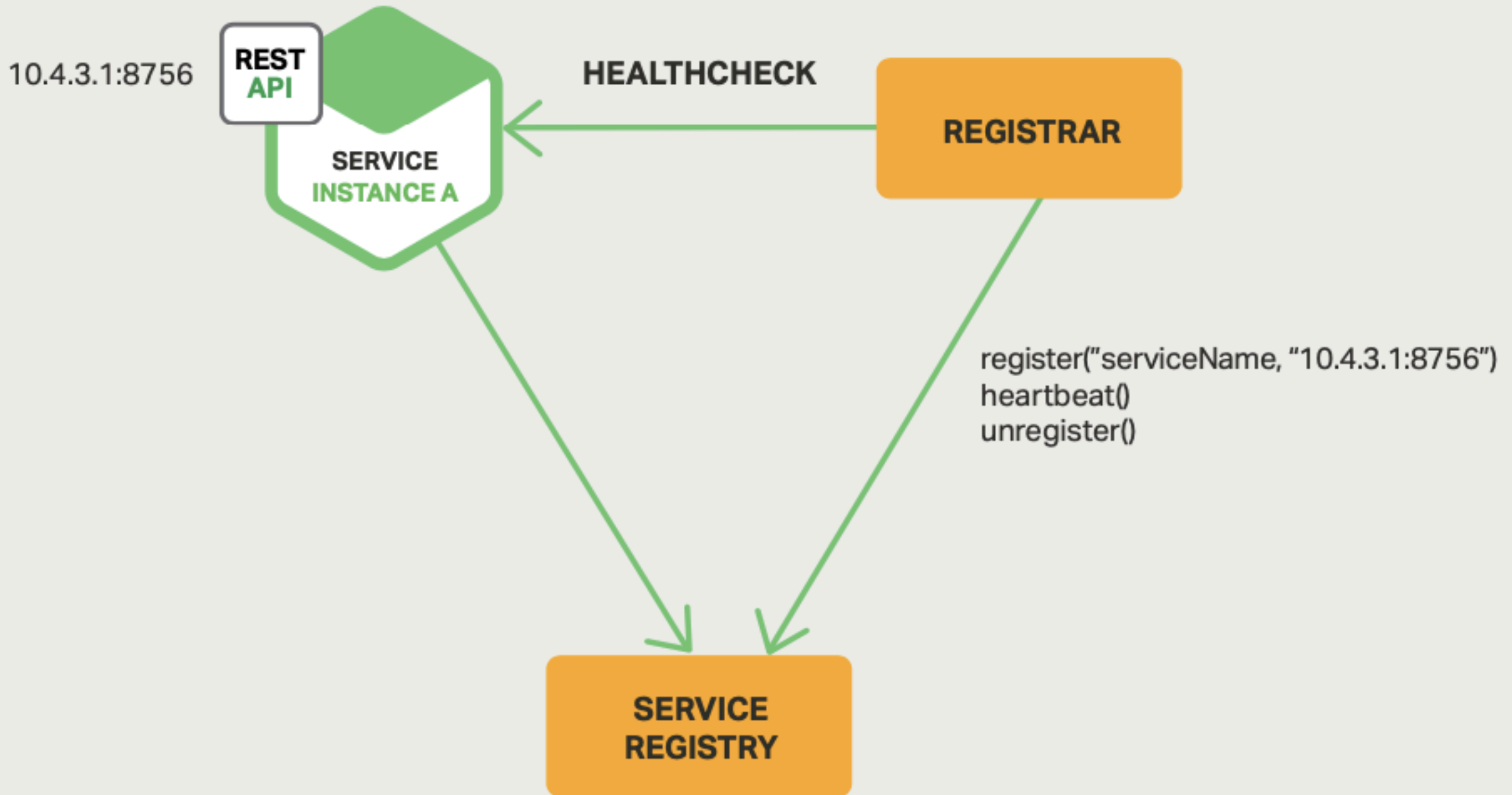
Self-Registration Pattern



Third-Party Registration Pattern

- When using the third-party registration pattern, **service instances aren't responsible for registering** themselves with the service registry.
- Instead, another system component known as the **service registrar** handles the registration.
- The service registrar **tracks changes** to the set of running instances by either **polling the deployment environment or subscribing to events**.
- When it notices a **newly available service instance**, it **registers** the instance with the service registry.
- The service registrar also **unregisters** terminated service instances.
- A major benefit is that **services are decoupled from the service registry**. You don't need to implement service-registration logic for each programming language and framework used by your developers. Instead, service instance registration is handled in a centralised manner within a dedicated service.
- One drawback of this pattern is that **unless it's built into the deployment environment**, it is **yet another highly available system component** that you need to set up and manage.

Third-Party Registration Pattern



{ e n d }

"The two most important days in your life are the day you are born and the day you find out why."

Mark Twain